CYLite Compiler Project

Language Specification and Grammar Modifications Report

Authors:

Tejas Lohia
[Shardul Junagade / 23110297]
[Swayam Borate / 23110066]

CS327 Compilers - Lab Assignment

March 30, 2026

Contents

1 Introduction **2**

2 Grammar Modifications **3**
2.1 Lexical Additions (Tokens) 3
2.2 The `elif` Conditional Construct 3
2.3 Native Error Handling: `try-except` 4
2.4 Ergonomic Iteration: `foreach` and `in range` 4
2.5 The explicit `pass` Operation 4
2.6 First-Class `print` Built-in 5
2.7 Results 5

3 Automaton Generation and Initial Analysis **6**
3.1 Bison Invocation and Outputs 6
3.2 Initial LALR(1) Automaton Analysis 6
3.3 Conflict Identification and Resolution 6
3.4 Shift/Reduce Conflict: The `ATOMIC` Keyword Ambiguity 7
3.5 Shift/Reduce Conflict: Abstract vs. Concrete Pointer Declarators 7
3.6 Shift/Reduce Conflicts: The Dangling-Else and `elif` Ambiguity 9
3.7 Shift/Reduce Conflicts: Propagation in `elif` Chains 9
3.8 Reduce/Reduce Conflict: The Comma Operator Ambiguity 10

1 Introduction

The landscape of programming languages has long been divided between highly performant, statically typed systems languages like C, and highly expressive, dynamically typed scripting languages like Python. C provides unparalleled control over hardware and memory layout, executing with optimal efficiency. However, its syntax can be unforgiving; error handling relies heavily on cumbersome return codes, control flow can become deeply nested and difficult to read, and iterative constructs often lead to off-by-one errors. Conversely, Python offers ergonomic syntax, explicit error handling blocks, and intuitive looping mechanisms, but at the cost of execution speed and strict compile-time type safety.

CYLite represents a modern hybridization of these two paradigms. It is designed as an extension of the YAPL (Yet Another Programming Language) baseline compiler, effectively injecting Python-inspired syntactic sugar and control flow structures into a C-style language grammar. The fundamental motivation behind CYLite is to provide developers with the readability and expressiveness of modern high-level languages without sacrificing the structural rigor of a statically compiled environment.

Developing a formal grammar for CYLite requires adapting these high-level programming concepts into a strictly unambiguous LALR(1) framework compatible with compiler generation tools like GNU Bison. By integrating these features directly at the lexical and grammatical level—rather than relying on standard library macros or external subroutines—the compiler pipeline can elevate them to first-class citizens in the language. This allows the compiler front-end to perform rigorous syntax checking, generate more optimized Abstract Syntax Trees (ASTs), and produce clearer compile-time error messages tailored to the programmer's explicit intent.

This project focused heavily on redefining the parser rules (`yapl.y`) to accommodate these new grammatical structures. By modifying the base YAPL grammar, we introduced several new nonterminal rules and shifted existing expressions to handle newly introduced keywords. Specifically, the grammar now natively understands chained conditional blocks (via `elif`), explicit null operations (via `pass`), native I/O primitives (`print`), ergonomic loop iterators (`foreach` and `in range`), and block-scoped error encapsulation (`try-except`).

This structural evolution ensures that CYLite can parse declarative flow states seamlessly while resolving dangling-else ambiguities and iteration pitfalls inherent to older grammars. This report breaks down the philosophy behind each addition and details the

exact grammatical transformations applied to achieve the CYLite language specification.

2 Grammar Modifications

To implement the CYLite language extensions, the lexical analyzer (generated via Flex) was first updated to recognize new keywords and allocate corresponding tokens. The parser (`yapl.y`) was subsequently heavily modified to integrate these new tokens into valid syntax trees.

2.1 Lexical Additions (Tokens)

We introduced the following terminal tokens into the parser to represent the core vocabulary of the new language features:

- `ELIF`, `PASS`, `TRY`, `EXCEPT`, `PRINT`, `RANGE`, `IN`, `FOREACH`

```
int ifs_wo_else=0;
int ladder_len=0,hold=0;
int max=-1;
%}

%token  ELIF PASS TRY EXCEPT PRINT RANGE IN FOREACH
%token  IDENTIFIER I_CONSTANT F_CONSTANT STRING_LITERAL FUNC_NAME SIZEOF
%token  PTR_OP INC_OP DEC_OP LEFT_OP RIGHT_OP LE_OP GE_OP EQ_OP NE_OP TH_OP
%token  AND_OP OR_OP MUL_ASSIGN DIV_ASSIGN MOD_ASSIGN ADD_ASSIGN
%token  SUB_ASSIGN LEFT_ASSIGN RIGHT_ASSIGN AND_ASSIGN
```

Figure 1: Implementation of the new CYLite tokens in the lexical analyzer.

2.2 The `elif` Conditional Construct

In standard C, chaining conditionals requires either deeply nested `else { if (...) }` blocks or the '`else if`' syntax, which is grammatically just an `else` block containing a single `if` statement. In complex logical branching, this can disrupt readability and complicate AST generation due to classic dangling-else ambiguities. Python's `elif` provides a unified, first-class construct for multi-way branching without artificial nesting.

Grammar Change: We defined a new nonterminal `elif_list` to explicitly handle zero or more `elif` branches recursively. We then expanded `selection_statement` to support an `if` block followed optionally by an `elif_list` and a final optional `else`.

```
elif_list
    : ELIF '(' expression ')' statement
    | elif_list ELIF '(' expression ')' statement
    ;

selection_statement
    : IF '(' expression ')' statement ELSE statement
    | IF '(' expression ')' statement
    | IF '(' expression ')' statement elif_list ELSE statement
    | IF '(' expression ')' statement elif_list
    ...
```

2.3 Native Error Handling: `try-except`

Error handling in C typically falls back to checking integer return codes after every function call, or using precarious `setjmp/longjmp` chains for global exits. Modern languages isolate fragile code within explicit boundaries, redirecting failures to localized catch blocks. By implementing `try-except` directly in the CYLite grammar, we establish dedicated scope blocks for execution and recovery, allowing the backend to map these to structured, object-level exception-handling routines.

Grammar Change: We introduced a `try_except_statement` combining the `TRY` keyword, a localized block (`compound_statement`), the `EXCEPT` keyword, and its associative block. This nonterminal was then injected into the overarching `statement` rule.

```
try_except_statement
    : TRY compound_statement EXCEPT compound_statement
    ;

statement
    ...
    | try_except_statement
```

2.4 Ergonomic Iteration: `foreach` and `in range`

Classic C `for` loops require boilerplate initialization, condition checking, and manual incrementation (e.g., `for(int i=0; i<10; i++)`). This pattern is verbose and inherently prone to off-by-one errors. CYLite adopts `foreach id in enumerable` and `for(id in range(a, b))` to abstract the iteration boilerplate. This makes the programmer's intent purely declarative rather than imperative, preventing out-of-bounds complications.

Grammar Change: The `iteration_statement` nonterminal was expanded to natively parse these specialized loops, tying identifiers directly to iterable sequences and mathematical ranges.

```
iteration_statement
    ...
    | FOR '(' IDENTIFIER IN RANGE '(' expression ',' expression ')' ')' statement
    | FOREACH '(' IDENTIFIER IN expression ')' compound_statement
```

2.5 The explicit `pass` Operation

In standard C, an intentionally empty block or stubbed function is written as an empty curly block {} or a lone semicolon `;`. This is visually ambiguous—compilers and semantic linters regularly flag it as a probable mistake. Promoting the explicit `pass` keyword provides an unambiguous syntactic placeholder, asserting that the empty block is intentional.

Grammar Change: `PASS ';'` was added directly as a terminal production alternative within the `statement` rule.

```
statement
    ...
    | PASS ';'
```

2.6 First-Class `print` Built-in

Rather than relying exclusively on importing `stdio.h` to use the variadic `printf` function—which requires cumbersome format strings and type awareness—CYLite promotes basic output to a native built-in language feature. By integrating it into the grammar, `print` can be more strictly type-checked or implicitly optimized for polymorphic output at compile-time by the parser's AST builder without requiring external linkage headers.

Grammar Change: The `print` construct was appended defensively to `postfix_expression`, supporting both empty calls and list arguments.

```
postfix_expression
    ...
    | PRINT '(' ')'
    | PRINT '(' argument_expression_list ')'
```

2.7 Results

The integration of these advanced features required careful consideration of LALR(1) parsing constraints to avoid shift/reduce and reduce/reduce conflicts within the Bison generator framework. By directly extending the `yapl.y` grammar definitions, CYLite successfully bridges the architectural gap between systems programming and scripting conveniences. Developers can now write highly readable, python-inspired code while leaning on the reliable, rigorous syntax-checking rules inherent to a C-like toolchain.

3 Automaton Generation and Initial Analysis

To construct the LALR(1) item sets and build the parser automaton (fulfilling Part 1 of the assignment requirements), we processed the modified `yapl.y` grammar file using GNU Bison.

3.1 Bison Invocation and Outputs

The parser generator was invoked using the following command:

```
$ bison -v -d yapl.y
```

This command utilizes two critical flags:

- **`-d`:** Generates the `yapl.tab.h` header file, which exposes the newly defined CYLite terminal tokens (e.g., `ELIF`, `FOREACH`, `TRY`) to the Flex lexical analyzer.
- **`-v` (Verbose):** Instructs Bison to generate a detailed `yapl.output` file. This file is crucial as it contains the human-readable representation of every LALR(1) item set and the complete state transition graph for the automaton.

3.2 Initial LALR(1) Automaton Analysis

The generation process successfully constructed the LALR(1) automaton required to parse the CYLite language specification. An initial analysis of the generated `yapl.output` report reveals the following structural metrics regarding the grammar's complexity:

- **Grammar Rules:** The parser implements over 130 distinct production rules to encapsulate both the core C-style expressions and the advanced CYLite extensions.
- **Item Sets (States):** The resulting LALR(1) automaton consists of 522 distinct states (item sets). This large state space is a direct result of the complex expression precedence and the newly introduced deeply nested control structures (such as `try-except` and `elif` ladders).

While the item sets and state machine were successfully compiled, the integration of optional branching (specifically the `elif` and `else` derivations) and ambiguous operator precedence introduced non-determinism into the parsing table. The initial Bison compilation reported exactly **6 shift/reduce conflicts** and **1 reduce/reduce conflict**. The identification, isolation, and resolution of these conflicts are documented in the following section.

3.3 Conflict Identification and Resolution

As noted in the initial automaton analysis, the introduction of CYLite's advanced control structures—specifically the `elif` ladders and optional `else` blocks—introduced non-determinism into the LALR(1) parsing table. When processed by GNU Bison, the grammar generated exactly 6 shift/reduce conflicts and 1 reduce/reduce conflict.

To satisfy the structural rigor required by the compiler front-end, we must isolate the specific item sets where the automaton encounters multiple valid transition paths, identify the conflicting productions, and apply deterministic precedence rules to resolve them.

```
State 27 conflicts: 1 shift/reduce
State 345 conflicts: 1 shift/reduce
State 473 conflicts: 2 shift/reduce
State 492 conflicts: 2 shift/reduce
State 515 conflicts: 1 reduce/reduce
```

Figure 2: Identification of the conflicting states in the LALR(1) automaton.

3.4 Shift/Reduce Conflict: The `ATOMIC` Keyword Ambiguity

One of the identified conflicts occurs in **State 27** and involves the `ATOMIC` keyword. This is a **Shift/Reduce conflict**.

Exact Productions Involved:

```
State 27

  159 atomic_type_specifier: ATOMIC • '(' type_name ')'
  163 type_qualifier: ATOMIC •

    '('  shift, and go to state 49

    '('       [reduce using rule 163 (type_qualifier)]
    $default  reduce using rule 163 (type_qualifier)
```

Figure 3: Example of a shift/reduce conflict involving the `ATOMIC` keyword.

Conflict Analysis: In this state, the parser has just consumed the `ATOMIC` token and is looking ahead at the next token, which is an opening parenthesis `'('`. The automaton faces a structural ambiguity:

- **Shift Action:** It can shift the `'('` token onto the stack, anticipating an `atomic_type_specifier` construct (e.g., `_Atomic(int)`).
- **Reduce Action:** It can immediately reduce the `ATOMIC` token into a `type_qualifier` (acting similarly to `const` or `volatile`), assuming the parenthesis belongs to a subsequent cast or expression.

Bison's default behavior resolves this by favoring the shift operation, but the conflict indicates a deeper grammatical overlap between qualifiers and type specifiers.

3.5 Shift/Reduce Conflict: Abstract vs. Concrete Pointer Declarators

Another significant conflict surfaces in **State 345**, representing a **Shift/Reduce conflict** driven by the overlap between named declarators and abstract type definitions, complicated by embedded semantic actions.

```
State 345

  169 declarator: pointer • $@1 direct_declarator
  202 abstract_declarator: pointer • direct_abstract_declarator
  203                      | pointer •

    '('  shift, and go to state 185
    '['  shift, and go to state 186

    IDENTIFIER  reduce using rule 168 ($@1)
    '('         [reduce using rule 168 ($@1)]
    $default    reduce using rule 203 (abstract_declarator)

    $@1                          go to state 134
    direct_abstract_declarator   go to state 313
```

Figure 4: Shift/Reduce conflict involving abstract and concrete pointer declarators in State 345.

Conflict Analysis: In this state, the parser has processed a pointer token (e.g., `*`) and is looking ahead at an opening parenthesis `'('`. The ambiguity arises because the parser cannot immediately determine if it is parsing an `abstract_declarator` (such as a typecast `(int *())`) or a concrete `declarator` (such as a variable declaration `int *(p)`).

Crucially, the concrete `declarator` rule contains an embedded mid-rule semantic action (Rule 168 / `$@1`), which increments the `pointer_decls` counter. The automaton faces two conflicting choices:

- **Shift Action:** Directly shift the `'('` onto the stack to begin constructing a `direct_abstract_decl` (which lacks the mid-rule action).
- **Reduce Action:** Immediately reduce the empty mid-rule action `$@1` (executing the counter increment) before shifting the `'('` to build a standard `direct_declarator`.

Bison defaults to shifting, which means pointer declarations wrapped in parentheses might temporarily bypass the embedded semantic counter until the conflict is explicitly resolved.

3.6 Shift/Reduce Conflicts: The Dangling-Else and `elif` Ambiguity

The most prominent structural ambiguities introduced by the CYLite extensions occur within the conditional branching logic, explicitly captured in **State 473**. This represents a classic **Shift/Reduce conflict** compounded by the new `elif` construct.

Exact Productions Involved:

```
State 473

  262 selection_statement: IF '(' expression ')' statement • ELSE $@2 statement
  263                    | IF '(' expression ')' statement •
  264                    | IF '(' expression ')' statement • elif_list ELSE statement
  265                    | IF '(' expression ')' statement • elif_list

    ELIF  shift, and go to state 490
    ELSE  shift, and go to state 491

    ELIF      [reduce using rule 263 (selection_statement)]
    ELSE      [reduce using rule 263 (selection_statement)]
    $default  reduce using rule 263 (selection_statement)

    elif_list  go to state 492
```

Figure 5: Shift/Reduce conflict involving the Dangling-Else and `elif` ambiguity in State 473.

Conflict Analysis: In State 473, the parser has successfully matched an entire `IF` statement up to its closing block (indicated by the dot `.` after `statement`). It is now looking ahead at the next token, which is either an `ELIF` or an `ELSE`. The automaton is trapped in a non-deterministic state due to the possibility of nested conditionals (e.g., `if (A) if (B) pass; else pass;`):

- **Shift Action:** The parser can shift the `ELIF` or `ELSE` token onto the stack, aggressively binding it to the most recently opened (innermost) `IF` statement.
- **Reduce Action:** The parser can immediately reduce the consumed tokens into a completed `selection_statement` (Rule 263). This action assumes the `ELIF` or `ELSE` token belongs to a parent (outer) `IF` block higher up in the syntax tree.

Because Bison resolves shift/reduce conflicts by favoring the shift action by default, it naturally binds `else` and `elif` clauses to the closest preceding `if`. However, explicit precedence directives are required to suppress the generator warnings and formalize this behavior.

3.7 Shift/Reduce Conflicts: Propagation in `elif` Chains

The dangling-else ambiguity documented in State 473 propagates directly into the evaluation of the recursive `elif_list` itself, as prominently visible in **State 492**. This is another **Shift/Reduce conflict**.

Exact Productions Involved:

Conflict Analysis: In State 492, the parser has successfully aggregated one or more `elif` branches into an `elif_list`. Upon encountering another `ELIF` or an `ELSE` token, it faces the same structural indecision as before:

```
State 492

  260 elif_list: elif_list • ELIF '(' expression ')' statement
  264 selection_statement: IF '(' expression ')' statement elif_list • ELSE statement
  265                    | IF '(' expression ')' statement elif_list •

    ELIF  shift, and go to state 502
    ELSE  shift, and go to state 503

    ELIF      [reduce using rule 265 (selection_statement)]
    ELSE      [reduce using rule 265 (selection_statement)]
    $default  reduce using rule 265 (selection_statement)
```

Figure 6: Shift/Reduce conflict in State 492 demonstrating propagation within `elif` chains.

- **Shift Action:** Shift the token to continue building the current `elif_list` (Rule 260) or to cap it off with an `ELSE` block (Rule 264).
- **Reduce Action:** Immediately reduce the entire `IF-ELIF` chain into a completed `selection_statement` (Rule 265), assuming the upcoming tokens belong to a parent `IF` construct higher in the hierarchy.

Just like State 473, this is resolved by enforcing precedence rules that explicitly instruct the parser to shift trailing conditionals rather than prematurely reducing the block.

3.8 Reduce/Reduce Conflict: The Comma Operator Ambiguity

The sole **Reduce/Reduce conflict** in the LALR(1) automaton surfaces in **State 515**, highlighting a structural overlap involving the comma `‘,‘` token.

Exact Productions Involved:

```
State 515

  89 expression: assignment_expression •
  90           | expression ',' assignment_expression •

    ','       reduce using rule 89 (expression)
    ','       [reduce using rule 90 (expression)]
    $default  reduce using rule 89 (expression)
```

Figure 7: Reduce/Reduce conflict in State 515 illustrating the comma operator ambiguity.

Conflict Analysis: This conflict arises from the dual syntactic role of the comma in CYLite. The comma can act as a sequence operator chaining multiple assignments together into a single unified expression (Rule 90), or it can act as a simple delimiter separating distinct elements in a list (such as function parameters or arguments for the newly added `print` built-in).

When the parser successfully reads an `assignment_expression` and encounters a lookahead comma `‘,‘`, it faces two equally valid reduction paths for the exact same sequence of tokens:

- **Reduce Action 1 (Rule 89):** Reduce the isolated `assignment_expression` into a base `expression`. The parser assumes the upcoming comma is a list separator (e.g., moving to the next argument in a function call).
- **Reduce Action 2 (Rule 90):** Reduce the entire parsed sequence (`expression ',' assignment_expression`) into a single chained `expression`. The parser assumes the comma is a mathematical sequence operator.

By default, Bison resolves Reduce/Reduce conflicts by blindly choosing the rule that appears first in the grammar file (Rule 89). However, relying on implicit file ordering is dangerous for compiler stability. To fix this, explicit left-associativity (`%left ','`) must be declared to instruct the parser on how to definitively group trailing commas.